

Advanced OOP Concepts in Python

A Comprehensive 23-Slide Presentation

Course: CS202 - Programming Systems

Student: Nguyen Quang Hieu - 25125043

Week: Week 6 — Assignment 2

Slide 1: Title & Cover

```
=====
                        ADVANCED OOP IN PYTHON
Multiple Inheritance, Interfaces, Inner Classes,
                        and Lambda Expressions
=====
```

- **Topic:** Advanced Object-Oriented Programming & Functional Paradigms in Python
 - **Target Audience:** Advanced Software Developers & Computer Science Students
 - **Environment:** Python 3.8+
-

Slide 2: Table of Contents

1. **Module 1: Multiple Inheritance & MRO**
 - Syntax & Mechanics
 - C3 Linearization Algorithm
 - The Diamond Problem & `super()`
2. **Module 2: Interfaces & Abstraction**
 - Duck Typing vs. Explicit Contracts
 - Abstract Base Classes (`abc` module)
 - Structural Subtyping with `typing.Protocol`
3. **Module 3: Inner & Nested Classes**
 - Definition & Scope Rules
 - Outer Instance Referencing
 - Design Patterns & Practical Use Cases
4. **Module 4: Lambda Expressions & Functional OOP**
 - Anonymous Functions & Syntax
 - Higher-Order Functions (`map`, `filter`, `sorted`)
 - Closure Pitfalls & Late Binding
5. **Module 5: Additional Advanced Python OOP Features**
 - Property Decorators, `@classmethod`, and `@staticmethod`
6. **Summary & Key Takeaways**

Slide 3: Module 1 — Multiple Inheritance in Python

What is Multiple Inheritance?

In Python, a derived class can inherit directly from more than one base class:

```
class BaseA:
    def method_a(self):
        print("BaseA")

class BaseB:
    def method_b(self):
        print("BaseB")

class Derived(BaseA, BaseB):
    pass
```

Key Capabilities

- **Code Reuse:** Combine capabilities from multiple independent hierarchies.
- **Mixins:** Small, focused helper classes added to extend a primary class's capabilities without complex class trees.

Slide 4: The Diamond Problem in Multiple Inheritance

The Classic Diamond Hierarchy

When two subclasses inherit from a common parent, and a fourth class inherits from both subclasses:

```
      [ Base ]
     /      \
  [ ChildA ] [ ChildB ]
     \      /
    [ GrandChild ]
```

The Conflict

If **Base** defines `speak()`, and both **ChildA** and **ChildB** override `speak()`, which version should **GrandChild** invoke when `grandchild.speak()` is called?

- In C++, this causes ambiguity unless virtual inheritance is used.
- In Python, this is strictly and deterministically resolved using **Method Resolution Order (MRO)**.

Slide 5: Method Resolution Order (MRO) & C3 Linearization

How Python Resolves Attribute Lookup

Python determines the search order for methods and attributes using the **C3 Linearization Algorithm**.

Key Rules of MRO:

1. **Children before Parents:** Derived classes are checked before their base classes.
2. **Left-to-Right Order:** Base classes are evaluated in the order specified in the class definition header
`class Child(Base1, Base2).`
3. **Monotonicity:** A class always appears before its parents across the entire inheritance tree.

Inspecting MRO in Python:

```
print(GrandChild.__mro__)  
# Output: (<class '__main__.GrandChild'>, <class '__main__.ChildA'>,  
#         <class '__main__.ChildB'>, <class '__main__.Base'>, <class 'object'>)
```

Slide 6: Cooperative Multiple Inheritance & `super()`

Why `super()` is Essential in Python

`super()` does **not** simply call the parent class. It delegates method calls to the **next class in the MRO chain**.

```
class Base:  
    def __init__(self):  
        print("Base init")  
  
class ChildA(Base):  
    def __init__(self):  
        super().__init__()  
        print("ChildA init")  
  
class ChildB(Base):  
    def __init__(self):  
        super().__init__()  
        print("ChildB init")  
  
class GrandChild(ChildA, ChildB):  
    def __init__(self):  
        super().__init__()  
        print("GrandChild init")
```

```
g = GrandChild()
```

Execution Output:

```
Base init
ChildB init
ChildA init
GrandChild init
```

Every class constructor is executed **exactly once** in a cooperative chain.

Slide 7: Code Demonstration — Python Mixins & MRO

```
class JSONSerializerMixin:
    def to_json(self):
        import json
        return json.dumps(self.__dict__)

class DatabaseModel:
    def __init__(self, id_val):
        self.id = id_val

    def save(self):
        print(f"Saving record {self.id} to DB")

class UserRecord(DatabaseModel, JSONSerializerMixin):
    def __init__(self, id_val, username):
        super().__init__(id_val)
        self.username = username

user = UserRecord(42, "alice")
user.save()           # Output: Saving record 42 to DB
print(user.to_json()) # Output: {"id": 42, "username": "alice"}
```

- **Benefit:** `JSONSerializerMixin` adds serialization capabilities without corrupting `DatabaseModel`'s inheritance hierarchy.

Slide 8: Module 2 — Interfaces in Python

Does Python have an `interface` keyword?

No. Python does not have a native **interface** keyword like Java or C#. Instead, Python supports interfaces through three distinct mechanisms:

1. **Duck Typing** ("If it walks like a duck and quacks like a duck...")
2. **Formal Abstract Base Classes** (**abc module**)
3. **Structural Subtyping** (**typing.Protocol**)

INTERFACE PATTERNS	
Dynamic / Informal	Duck Typing
Formal Inheritance	abc.ABC & @abstractmethod
Static / Structural Typing	typing.Protocol

Slide 9: Duck Typing vs. Abstract Base Classes (**abc**)

Duck Typing (Informal Interface)

```
class FileLogger:
    def log(self, message: str):
        print(f"File log: {message}")

class ConsoleLogger:
    def log(self, message: str):
        print(f"Console log: {message}")

def execute_logging(logger):
    logger.log("System startup") # Works as long as logger has a log() method
```

Limitations of Duck Typing:

- No compile-time or instantiation-time enforcement.
- Missing methods only fail at runtime when invoked.

Slide 10: Abstract Base Classes (**abc module**)

Enforcing Contracts with **abc.ABC**

The **abc** module provides **ABC** and **@abstractmethod** to enforce that subclasses implement required methods.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float:
        """Calculate area of shape"""
        pass

    @abstractmethod
    def perimeter(self) -> float:
        """Calculate perimeter of shape"""
        pass

class Circle(Shape):
    def __init__(self, radius: float):
        self.radius = radius

    def area(self) -> float:
        return 3.14159 * self.radius ** 2

# ERROR if perimeter() is not implemented:
# TypeError: Can't instantiate abstract class Circle without an implementation
# for abstract method 'perimeter'
```

Slide 11: Structural Subtyping with `typing.Protocol`

Static Interface Checking without Inheritance (Python 3.8+)

`typing.Protocol` allows explicit interface definition that works with static type checkers (`mypy`) **without needing explicit class inheritance**.

```
from typing import Protocol

class Printable(Protocol):
    def print_details(self) -> None:
        ...

class Invoice: # Notice: Invoice does NOT inherit from Printable!
    def print_details(self) -> None:
        print("Invoice #1001: $250.00")

def render(item: Printable) -> None:
    item.print_details()

render(Invoice()) # Validated statically by type checkers!
```

Slide 12: Module 3 — Inner Classes & Nested Classes in Python

What is an Inner / Nested Class?

An inner class is a class defined directly inside the body of another enclosing class.

```
class Outer:
    def __init__(self, name):
        self.name = name

    class Inner:
        def __init__(self, outer_ref):
            self.outer_ref = outer_ref

        def display(self):
            print(f"Inner accessing Outer name: {self.outer_ref.name}")
```

Slide 13: Scope & Access Rules for Python Inner Classes

Crucial Difference: Python vs. Java Inner Classes

- In Java, an inner class instance **automatically holds a reference** to its outer class instance (`Outer.this`).
- In Python, an inner class is **simply an attribute of the outer class namespace**. It does **NOT** automatically receive a reference to the outer class instance!

Passing the Outer Instance Explicitly:

```
class Computer:
    def __init__(self, brand, cpu_model):
        self.brand = brand
        # Pass self to inner class constructor:
        self.cpu = self.CPU(self, cpu_model)

    class CPU:
        def __init__(self, computer_inst, model):
            self.computer_inst = computer_inst
            self.model = model

        def get_specs(self):
            return f"{self.computer_inst.brand} powered by {self.model}"
```

```
c = Computer("Dell", "Intel i9")
print(c.cpu.get_specs()) # Dell powered by Intel i9
```

Slide 14: Practical Use Cases for Inner Classes in Python

1. Logical Grouping & Encapsulation

When a helper class is tightly coupled to a single main class and has no utility elsewhere.

2. Builder Pattern

Using a nested `Builder` class to construct complex configuration objects.

```
class DatabaseConnection:
    def __init__(self, host, port, timeout):
        self.host = host
        self.port = port
        self.timeout = timeout

    class Builder:
        def __init__(self):
            self.host = "localhost"
            self.port = 5432
            self.timeout = 30

        def set_host(self, host):
            self.host = host
            return self

        def build(self):
            return DatabaseConnection(self.host, self.port, self.timeout)

db = DatabaseConnection.Builder().set_host("db.example.com").build()
```

Slide 15: Module 4 — Lambda Expressions in Python

What is a Lambda Expression?

A `lambda` expression is a small, anonymous function defined inline using the `lambda` keyword.

Syntax:

`lambda arguments: expression`

```
# Standard function
def add_std(x, y):
    return x + y

# Lambda equivalent
add_lambda = lambda x, y: x + y

print(add_std(3, 5))    # 8
print(add_lambda(3, 5)) # 8
```

Key Limitations of Python Lambdas:

- Must consist of a **single expression**.
- Cannot contain multi-line statements, loops, or assignments (=).

Slide 16: Lambdas with Higher-Order Functions

`map()`, `filter()`, and `sorted()`

Lambdas are commonly passed as short callback arguments into higher-order functions.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# 1. filter(): Filter even numbers
evens = list(filter(lambda x: x % 2 == 0, numbers))
# Result: [2, 4, 6, 8, 10]

# 2. map(): Square every number
squares = list(map(lambda x: x ** 2, numbers))
# Result: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

# 3. sorted(): Sort list of tuples by second element
students = [("Alice", 88), ("Bob", 95), ("Charlie", 78)]
sorted_students = sorted(students, key=lambda s: s[1], reverse=True)
# Result: [('Bob', 95), ('Alice', 88), ('Charlie', 78)]
```

Slide 17: Closures & Late Binding Pitfall in Python Lambdas

The Late Binding Gotcha

In Python, variables captured in closures are looked up at **invocation time**, not definition time!

```
# BUGGY CODE:
funcs = [lambda: i for i in range(3)]
for f in funcs:
    print(f())
# Expected: 0, 1, 2
# Actual Output: 2, 2, 2 (because 'i' is looked up when f() is called!)
```

The Solution: Default Parameter Binding

```
# CORRECT CODE: Bind 'i' at definition time via default parameter
funcs = [lambda i=i: i for i in range(3)]
for f in funcs:
    print(f())
# Output: 0, 1, 2
```

Slide 18: Module 5 — Additional Advanced Python OOP Features

1. `@property`, `@setter`, and `@deleter`

Provides clean encapsulation without altering public attribute access syntax.

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("Balance cannot be negative")
        self._balance = value

acc = Account(100)
acc.balance = 150 # Triggers setter validation automatically!
```

Slide 19: `@classmethod` vs. `@staticmethod` vs.

Instance Methods

```
class MathUtils:
    factor = 2 # Class attribute

    def __init__(self, val):
        self.val = val

    # 1. Instance Method: Receives 'self' (instance state)
    def compute(self):
        return self.val * MathUtils.factor

    # 2. Class Method: Receives 'cls' (class state, alternative constructor)
    @classmethod
    def from_string(cls, str_val):
        return cls(int(str_val))

    # 3. Static Method: Isolated utility (receives no implicit self/cls)
    @staticmethod
    def is_positive(num):
        return num > 0

m = MathUtils.from_string("10")
print(m.compute())           # Output: 20
print(MathUtils.is_positive(5)) # Output: True
```

Slide 20: Special (Magic/Dunder) Methods in OOP

Python allows custom objects to integrate natively with built-in syntax via dunder methods:

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    # String representation for developers
    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

    # Operator overloading for '+'
    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    # Callable instance (__call__)
    def __call__(self, scale):
        return Vector(self.x * scale, self.y * scale)
```

```
v1 = Vector(2, 3)
v2 = Vector(4, 1)
v3 = v1 + v2
print(v3)          # Vector(6, 4)
print(v3(2))       # Vector(12, 8)
```

Slide 21: Python Metaclasses (Brief Overview)

"Classes of Classes"

In Python, classes themselves are instances of metaclasses (`type`).

```
# Custom Metaclass enforcing upper-case class attributes
class EnforceUppercaseMeta(type):
    def __new__(cls, name, bases, dct):
        uppercase_dct = {}
        for key, val in dct.items():
            if not key.startswith("__") and not key.isupper():
                raise TypeError(f"Attribute '{key}' in class '{name}' must be
UPPERCASE!")
            uppercase_dct[key] = val
        return super().__new__(cls, name, bases, uppercase_dct)

# Class creation will fail if rules are violated
class Config(metaclass=EnforceUppercaseMeta):
    API_KEY = "12345" # Valid
    # db_host = "localhost" # Would raise TypeError!
```

Slide 22: Comparison Summary Matrix

Concept	Python Implementation	Main Advantage	Potential Pitfall
Multiple Inheritance	<code>class C(A, B):</code> with C3 MRO	High modularity & Mixins	Diamond problem if <code>super()</code> is misused
Interfaces	<code>abc.ABC</code> , <code>typing.Protocol</code>	Explicit contracts & type checking	Over-engineering simple scripts
Inner Class	Class defined inside class body	Namespace grouping	No automatic reference to <code>outer</code> instance
Lambda	<code>lambda args: expr</code>	Concise inline callbacks	Restricted to single-line expressions

Slide 23: Conclusion & Key Takeaways

1. **Multiple Inheritance in Python** is safe and predictable thanks to **C3 Linearization (MRO)** and cooperative `super()` calls.
 2. **Interfaces** in modern Python range from flexible **Duck Typing** to formal **Abstract Base Classes (abc)** and static **Protocols (`typing.Protocol`)**.
 3. **Inner Classes** in Python act as namespace scopes and require explicit passing of the outer instance if access is needed.
 4. **Lambda Expressions** excel at short inline operations with higher-order functions (`map`, `filter`, `sorted`), but care must be taken with closure late binding.
-

End of Presentation — Assignment 2 (Python)